You said:

```
Test the search feature.
```

ChatGPT said:

Sure — I can help test the search feature. Please provide the **search term/query** you want to test (for example, a topic, website, document, or keyword).

If you mean testing a **search box in an application/website**, tell me which app or share the details, and I'll guide you through the test steps.

You said:

Act as a Senior QA Automation Engineer. Generate a comprehensive end-to-end test case document for testing the search feature of an enterprise e-commerce web application. Include functional scenarios (keyword search, filters, sorting), negative scenarios (special characters, empty input, SQL injection), and performance/UI scenarios (search load times, auto-suggestions). Provide the output in a structured Markdown table with columns: Test ID, Scenario, Test Steps, Test Data, Expected Result, and Priority.

ChatGPT said:

End-to-End Test Cases — Enterprise E-Commerce Search Feature

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
SEARCH_001	Verify search box availability on homepage	1. Launch e-commerce application. 2. Navigate to homepage. 3. Locate search bar.	N/A	Search bar should be visible, enabled, and accessible to users.	High
SEARCH_002	Verify keyword search with valid product name	1. Enter a valid product keyword. 2. Click Search icon or press Enter. 3. Observe results.	Keyword: "Laptop"	Relevant products containing "Laptop" should be displayed. Search results count and product details should load correctly.	High

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
SEARCH_003	Verify search using exact product name	1. Enter complete product name. 2. Execute search. 3. Validate result accuracy.	"Apple MacBook Pro 14"	Exact matching product should appear at the top of search results.	High
SEARCH_004	Verify partial keyword search	1. Enter partial product keyword. 2. Submit search. 3. Verify results.	"Mac"	System should display products matching partial keyword based on search configuration.	High
SEARCH_005	Verify search using multiple keywords	1. Enter multiple related keywords. 2. Perform search. 3. Validate results.	"Wireless Bluetooth Headphones"	Results should contain products matching all or relevant keywords.	High
SEARCH_006	Verify case-insensitive search	1. Search using lowercase keyword. 2. Search using uppercase keyword. 3. Compare results.	"iphone", "IPHONE"	Search results should be identical regardless of keyword case.	Medium
SEARCH_007	Verify search with leading/trailing spaces	1. Enter keyword with spaces before and after. 2. Execute search. 3. Check results.	" Laptop "	System should trim unnecessary spaces and return correct results.	Medium
SEARCH_008	Verify search for unavailable product	1. Enter a product keyword that does not exist. 2. Execute search.	"XYZProduct12345"	User should see "No results found" message with helpful suggestions.	High
SEARCH_009	Verify search result relevance ranking	1. Search for common keyword. 2. Review first page results.	"Phone"	Most relevant and popular products should appear according to business ranking rules.	High

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
SEARCH_010	Verify search result pagination	1. Perform search returning multiple pages. 2. Navigate between pages.	"Shoes"	Pagination should work correctly without duplicate or missing products.	Medium
SEARCH_011	Verify search with category filter	1. Search for product keyword. 2. Apply category filter. 3. Validate results.	Keyword: "Shirt" Category: Men's Clothing	Only products from selected category should be displayed.	High
SEARCH_012	Verify search with price range filter	1. Search product. 2. Apply minimum and maximum price filters. 3. Validate results.	Product: "Laptop" Price: $500-$1000	Only products within selected price range should appear.	High
SEARCH_013	Verify search with brand filter	1. Search product. 2. Select brand filter. 3. Verify results.	Product: "Mobile" Brand: Apple	Results should contain only selected brand products.	High
SEARCH_014	Verify multiple filters combination	1. Search product. 2. Apply category, brand, and price filters. 3. Verify filtered results.	Laptop + Dell + $500-$1200	Results should satisfy all applied filter conditions.	High
SEARCH_015	Verify removing applied filters	1. Apply filters. 2. Remove one or all filters. 3. Observe results.	Brand filter removal	Removed filters should no longer affect search results.	Medium
SEARCH_016	Verify sorting by price low to high	1. Perform product search. 2. Select "Price Low to High".	"Laptop"	Products should be displayed in ascending price order.	High

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
SEARCH_017	Verify sorting by price high to low	1. Perform search. 2. Select "Price High to Low".	"Laptop"	Products should be displayed in descending price order.	High
SEARCH_018	Verify sorting by popularity	1. Search products. 2. Select popularity sorting.	"Smartphone"	Most popular products should appear first.	Medium
SEARCH_019	Verify sorting by customer rating	1. Search products. 2. Select rating sort option.	"Headphones"	Products should be sorted based on customer ratings.	Medium
SEARCH_020	Verify auto-suggestion functionality	1. Click search box. 2. Start typing keyword. 3. Observe suggestions.	"Sam"	Relevant product/category suggestions should appear dynamically.	High
SEARCH_021	Verify auto-suggestion selection	1. Enter partial keyword. 2. Select suggestion. 3. Verify navigation.	"Nike" suggestion	User should be redirected to relevant search results.	High
SEARCH_022	Verify auto-suggestion response time	1. Enter characters slowly. 2. Measure suggestion loading time.	"Laptop"	Suggestions should appear within acceptable response time (e.g., <500 ms).	High
SEARCH_023	Verify search using keyboard Enter key	1. Enter keyword. 2. Press Enter instead of clicking button.	"Camera"	Search should execute successfully.	Medium
SEARCH_024	Verify search button functionality	1. Enter keyword. 2. Click search icon/button.	"Watch"	Search should execute and display results.	Medium

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
SEARCH_025	Verify search on mobile responsive UI	1. Open website on mobile device/browser. 2. Perform search.	"Shoes"	Search feature should work correctly on mobile UI.	Medium
SEARCH_026	Verify empty search submission	1. Leave search box empty. 2. Click Search button.	Blank input	System should display validation message or prevent empty search execution.	High
SEARCH_027	Verify search with only spaces	1. Enter spaces only. 2. Submit search.	" "	System should treat input as empty and show validation message.	High
SEARCH_028	Verify search with special characters	1. Enter special characters. 2. Execute search.	"@#$%^&*"	Application should handle input safely and display no-result/validation message.	High
SEARCH_029	Verify search with unsupported characters	1. Enter unsupported Unicode characters. 2. Execute search.	"你好@@@"	Application should not crash and should handle input gracefully.	Medium
SEARCH_030	Verify SQL injection prevention in search	1. Enter SQL injection payload. 2. Execute search. 3. Monitor response.	`' OR '1'='1`	Application should reject malicious input and prevent database exposure.	Critical
SEARCH_031	Verify XSS attack prevention	1. Enter JavaScript payload. 2. Execute search. 3. Check page behavior.	`<script>alert('XSS')</script>`	Script should not execute. Input should be sanitized.	Critical

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
SEARCH_032	Verify extremely long search input	1. Paste large text into search box. 2. Execute search.	10,000-character string	Application should handle input without crash or performance degradation.	High
SEARCH_033	Verify search performance with normal load	1. Perform multiple searches. 2. Measure response time.	Common keywords	Search results should load within defined SLA (example: <2 seconds).	High
SEARCH_034	Verify search performance under peak load	1. Simulate multiple concurrent users. 2. Execute searches simultaneously. 3. Monitor response.	1000 concurrent users	Search should remain available with acceptable response time and error rate.	Critical
SEARCH_035	Verify search API response time	1. Capture search API request. 2. Measure API response duration.	Search API endpoint	API should respond within agreed performance threshold.	High
SEARCH_036	Verify search behavior during network interruption	1. Start search. 2. Disconnect network. 3. Observe behavior.	Network unavailable	User should receive meaningful error message instead of application crash.	Medium
SEARCH_037	Verify search error handling when backend service is unavailable	1. Disable search service. 2. Perform search.	Backend unavailable	Application should show graceful error message and recovery option.	High
SEARCH_038	Verify browser compatibility	1. Open application in Chrome, Edge, Firefox, Safari. 2. Perform search.	Multiple browsers	Search should function consistently across supported browsers.	Medium

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
SEARCH_039	Verify accessibility of search feature	1. Navigate using keyboard only. 2. Use screen reader tools. 3. Test labels.	Keyboard navigation	Search should meet accessibility standards (ARIA labels, focus handling).	Medium
SEARCH_040	Verify search history functionality (if available)	1. Perform multiple searches. 2. Open search history.	Previous searches	User should see recent searches according to configured rules.	Low
SEARCH_041	Verify search results after product inventory update	1. Update product availability. 2. Perform search. 3. Validate results.	Out-of-stock product	Search results should reflect latest inventory status.	High
SEARCH_042	Verify search analytics tracking	1. Perform searches. 2. Check analytics events/logs.	"Laptop"	Search events should be captured correctly for reporting.	Medium
SEARCH_043	Verify typo tolerance/spell correction	1. Enter misspelled keyword. 2. Execute search.	"iphon"	System should suggest corrected spelling or relevant products.	Medium
SEARCH_044	Verify search result UI consistency	1. Perform search. 2. Review result page layout.	Any keyword	Product cards, images, prices, and buttons should display correctly.	High
SEARCH_045	Verify search results persistence after refresh	1. Perform search. 2. Refresh browser page.	"Laptop"	Search state should persist or behave according to application design.	Low

Suggested Automation Coverage

Area	Automation Priority	Recommended Tools
Keyword search validation	High	Selenium / Playwright + TestNG/JUnit

Area	Automation Priority	Recommended Tools
Filters & sorting	High	Selenium / Cypress
API search validation	Critical	Rest Assured / Postman/Newman
SQL Injection & XSS security tests	Critical	OWASP ZAP / Burp Suite
Performance testing	Critical	JMeter / Gatling
Cross-browser validation	Medium	Selenium Grid / BrowserStack
Auto-suggestion testing	High	Playwright/Selenium with explicit waits
Accessibility testing	Medium	Axe DevTools

Total Test Scenarios: 45
Coverage Areas: Functional + Negative + Security + Performance + UI + Accessibility + Automation Readiness.

You said:

```
Test the cart functionality."
```

ChatGPT said:

End-to-End Test Cases — Enterprise E-Commerce Shopping Cart Functionality

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_001	Verify cart icon availability	1. Launch e-commerce application. 2. Navigate to homepage. 3. Locate cart icon.	N/A	Cart icon should be visible, enabled, and accessible.	High

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_002	Verify adding a product to cart	1. Login to application. 2. Search for a product. 3. Open product details. 4. Click "Add to Cart". 5. Open cart.	Product: iPhone 16	Product should be added successfully and displayed in cart.	Critical
CART_003	Verify adding multiple products to cart	1. Add different products one by one. 2. Open cart.	Laptop + Headphones + Mouse	All selected products should appear in cart with correct details.	High
CART_004	Verify adding same product multiple times	1. Add a product to cart. 2. Add the same product again.	Product: Wireless Headphones	System should increase quantity or show duplicate item based on business rules.	High
CART_005	Verify product details in cart	1. Add product to cart. 2. View cart details.	Product: Laptop	Product name, image, SKU, price, quantity, and availability should display correctly.	High
CART_006	Verify cart quantity update	1. Add product to cart. 2. Increase quantity using quantity control. 3. Observe cart.	Quantity: 1 → 3	Quantity should update successfully and totals should recalculate.	Critical
CART_007	Verify decreasing product quantity	1. Add multiple quantities. 2. Reduce quantity.	Quantity: 5 → 2	Quantity should decrease correctly and subtotal should update.	High
CART_008	Verify removing product from cart	1. Add product. 2. Click Remove/Delete option.	Product: Shoes	Product should be removed from cart successfully.	Critical
CART_009	Verify removing all products from cart	1. Add multiple products. 2. Remove each item.	Multiple products	Cart should become empty and display empty cart message.	Medium

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_010	Verify empty cart state	1. Open cart without adding products.	Empty cart	System should display appropriate empty cart message and continue shopping option.	High
CART_011	Verify cart item count badge	1. Add product to cart. 2. Check cart badge count.	Add 3 products	Cart count should display correct number of items.	High
CART_012	Verify cart persistence after logout/login	1. Add product. 2. Logout. 3. Login again. 4. Open cart.	Registered user	Cart items should persist according to application rules.	High
CART_013	Verify guest user cart persistence	1. Add product without login. 2. Refresh browser. 3. Open cart.	Guest user	Cart should retain items using session/cookie mechanism.	Medium
CART_014	Verify cart persistence across devices	1. Add item on Device A. 2. Login on Device B. 3. Open cart.	Same user account	Cart should sync across devices if supported.	Medium
CART_015	Verify cart quantity limit	1. Add product. 2. Increase quantity beyond allowed limit.	Quantity: 999	System should restrict quantity and display proper validation message.	High
CART_016	Verify adding out-of-stock product	1. Open unavailable product. 2. Attempt to add to cart.	Out-of-stock item	Add to cart should be disabled or proper error message displayed.	High
CART_017	Verify inventory validation in cart	1. Add product. 2. Change inventory from backend. 3. Open cart.	Product stock reduced	Cart should notify user about stock availability changes.	High

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_018	Verify price calculation in cart	1. Add products. 2. Check subtotal.	Product A: $100 Quantity: 2	Subtotal should calculate correctly ($200).	Critical
CART_019	Verify total amount calculation	1. Add multiple products. 2. Check grand total.	Products with different prices	Total should equal item subtotal + taxes + shipping - discounts.	Critical
CART_020	Verify tax calculation	1. Add taxable product. 2. Proceed to cart.	Product: Electronics	Tax amount should calculate according to configured tax rules.	High
CART_021	Verify shipping charge calculation	1. Add product. 2. Select delivery option. 3. Check charges.	Standard Shipping	Shipping cost should display correctly.	High
CART_022	Verify free shipping eligibility	1. Add products above free shipping threshold. 2. Check shipping cost.	Cart value > $100	Shipping charge should become free if applicable.	Medium
CART_023	Verify discount coupon application	1. Add product. 2. Enter valid coupon. 3. Apply coupon.	Coupon: SAVE20	Discount should apply and total should update.	Critical
CART_024	Verify invalid coupon handling	1. Add product. 2. Enter invalid coupon. 3. Apply.	Coupon: INVALID123	System should display invalid coupon error message.	High
CART_025	Verify expired coupon handling	1. Apply expired coupon.	Coupon: EXPIRED20	System should reject expired coupon.	Medium
CART_026	Verify removing applied coupon	1. Apply coupon. 2. Remove coupon.	Valid coupon	Discount should be removed and total recalculated.	Medium

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_027	Verify save-for-later functionality	1. Add product. 2. Move item to wishlist/save later.	Product: Watch	Product should move successfully.	Medium
CART_028	Verify moving wishlist item to cart	1. Open wishlist. 2. Click Add to Cart.	Saved product	Product should be added to cart successfully.	High
CART_029	Verify cart sorting/display order	1. Add multiple products. 2. View cart.	Multiple products	Products should display according to configured order.	Low
CART_030	Verify cart page responsiveness	1. Open cart on desktop/tablet/mobile. 2. Check layout.	Multiple devices	Cart UI should adapt correctly without overlapping elements.	Medium
CART_031	Verify cart UI elements	1. Open cart page. 2. Validate buttons and labels.	N/A	Quantity controls, remove button, checkout button should display correctly.	High
CART_032	Verify checkout navigation from cart	1. Add product. 2. Click Checkout.	Product in cart	User should navigate to checkout page.	Critical
CART_033	Verify cart data during checkout	1. Add products. 2. Proceed to checkout.	Product details	Cart items, quantities, and prices should match checkout summary.	Critical
CART_034	Verify cart session timeout behavior	1. Add products. 2. Wait until session expires. 3. Open cart.	Session timeout	System should handle expired session gracefully.	Medium
CART_035	Verify browser refresh behavior	1. Add products. 2. Refresh cart page.	Product in cart	Cart data should remain intact.	High

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_036	Verify back browser navigation	1. Add product. 2. Navigate away. 3. Click browser back.	Product in cart	Cart state should remain consistent.	Medium
CART_037	Verify special character handling in coupon field	1. Enter special characters in coupon input. 2. Apply.	`@#$%^`	System should validate input safely.	Medium
CART_038	Verify SQL injection prevention in cart inputs	1. Enter SQL payload in coupon/search-related cart fields. 2. Submit.	`' OR 1=1 --`	Application should reject malicious input and prevent database attacks.	Critical
CART_039	Verify XSS prevention in cart fields	1. Enter script payload. 2. Submit.	`<script>alert(1)</script>`	Script should not execute. Input should be sanitized.	Critical
CART_040	Verify cart performance under normal load	1. Add multiple products. 2. Load cart page. 3. Measure response time.	50 cart items	Cart should load within defined SLA (example: <3 seconds).	High
CART_041	Verify cart performance under peak load	1. Simulate multiple users adding products. 2. Monitor system response.	10,000 concurrent users	Cart service should remain stable without failures.	Critical
CART_042	Verify cart API response time	1. Capture cart API calls. 2. Measure response time.	Add/update/remove APIs	APIs should respond within agreed performance threshold.	High
CART_043	Verify cart recovery after network failure	1. Add product. 2. Disconnect network. 3. Retry cart operation.	Network interruption	User should receive meaningful error message and recover operation.	Medium

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_044	Verify browser compatibility	1. Test cart in Chrome, Edge, Firefox, Safari.	Supported browsers	Cart functionality should work consistently.	Medium
CART_045	Verify accessibility of cart functionality	1. Navigate using keyboard. 2. Test with screen reader.	Keyboard navigation	Cart should support accessibility standards.	Medium

Automation Coverage Recommendation

Area	Priority	Recommended Tools
Add/remove/update cart items	Critical	Selenium / Playwright / Cypress
Cart API validation	Critical	Rest Assured / Postman Newman
Price & discount calculation	Critical	API + UI automation
Checkout integration	Critical	End-to-end automation
Security testing (SQL Injection/XSS)	Critical	OWASP ZAP / Burp Suite
Performance testing	Critical	JMeter / Gatling
Cross-browser testing	Medium	Selenium Grid / BrowserStack
Responsive UI testing	Medium	Playwright Mobile Emulation

Total Test Scenarios: 45
Coverage Areas: Functional + Negative + Security + Performance + UI + Accessibility + Automation Readiness.

You said:

Act as a Lead Software Quality Assurance Analyst. Write a detailed test scenario suite for testing the shopping cart functionality of a high-volume online retail platform. Cover item addition, deletion, and quantity updates, price and tax calculation accuracy, coupon/discount

code application, and out-of-stock item handling. Structure the output into clear categories using numbered lists.Provide the output in a structured Markdown table with columns: Test ID, Scenario, Test Steps, Test Data, Expected Result, and Priority."

ChatGPT said:

Shopping Cart Functionality — Comprehensive Test Scenario Suite

Role: Lead Software Quality Assurance Analyst
Application Type: High-Volume Enterprise Online Retail Platform

---

1. Item Addition Scenarios

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_ADD_001	Verify adding a single product to cart	1. Login to application. 2. Search for a valid product. 3. Open product details page. 4. Click **Add to Cart**. 5. Navigate to cart.	Product: Wireless Headphones	Product should be successfully added to cart with correct name, image, price, and quantity.	Critical
CART_ADD_002	Verify adding multiple different products to cart	1. Search for Product A. 2. Add Product A to cart. 3. Search for Product B. 4. Add Product B to cart. 5. Open cart.	Laptop + Mouse + Keyboard	All selected products should appear in cart with correct details.	High

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_ADD_003	Verify adding the same product multiple times	1. Add product to cart. 2. Return to product page. 3. Add same product again.	Product: Smartphone	System should increase quantity or handle duplicate item according to business rules.	High
CART_ADD_004	Verify adding product from search results page	1. Search for product. 2. Click Add to Cart directly from results page.	Product: Running Shoes	Product should be added without requiring product details page navigation.	Medium
CART_ADD_005	Verify adding product with selectable options	1. Select product variant. 2. Select size/color/storage. 3. Add to cart.	T-Shirt: Size M, Blue	Selected variant details should be displayed correctly in cart.	High
CART_ADD_006	Verify cart count update after adding item	1. Add product to cart. 2. Observe cart badge count.	Add 3 products	Cart count should increase accurately.	High
CART_ADD_007	Verify guest user adding items to cart	1. Open website without login. 2. Add product. 3. Open cart.	Guest user + Product	Product should be added and maintained in session.	Medium
CART_ADD_008	Verify adding maximum allowed quantity	1. Add product. 2. Increase quantity to maximum allowed limit.	Quantity Limit: 10	System should allow valid maximum quantity.	High

2. Item Deletion / Removal Scenarios

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_DEL_001	Verify removing a product from cart	1. Add product to cart. 2. Click Remove/Delete option. 3. Refresh cart.	Product: Laptop	Product should be removed successfully.	Critical
CART_DEL_002	Verify removing one item from multiple cart items	1. Add multiple products. 2. Remove one product. 3. Verify remaining products.	Laptop + Phone + Watch	Only selected product should be removed. Other items should remain unchanged.	High
CART_DEL_003	Verify removing all items from cart	1. Add multiple products. 2. Remove each product.	Multiple products	Cart should display empty cart state.	Medium
CART_DEL_004	Verify undo/remove recovery functionality	1. Remove product. 2. Click Undo if available.	Removed product	Product should be restored to cart.	Medium
CART_DEL_005	Verify cart total recalculation after deletion	1. Add multiple products. 2. Note total amount. 3. Remove one product.	Product A + Product B	Total amount should update correctly after removal.	Critical
CART_DEL_006	Verify deleting unavailable product from cart	1. Add product. 2. Make product unavailable from inventory. 3. Remove product.	Out-of-stock product	Product should be removed successfully without errors.	High

3. Quantity Update Scenarios

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_QTY_001	Verify increasing product quantity	1. Add product to cart. 2. Click increase quantity button. 3. Observe cart.	Quantity: 1 → 3	Quantity should increase and subtotal should update correctly.	Critical
CART_QTY_002	Verify decreasing product quantity	1. Add multiple quantities. 2. Reduce quantity.	Quantity: 5 → 2	Quantity should decrease correctly.	High
CART_QTY_003	Verify quantity cannot be reduced below minimum	1. Add single product. 2. Click decrease quantity.	Quantity: 1	System should prevent quantity below minimum or remove item based on rules.	High
CART_QTY_004	Verify manual quantity entry	1. Enter quantity manually. 2. Click update cart.	Quantity: 20	Quantity should update after validation.	High
CART_QTY_005	Verify invalid quantity input handling	1. Enter negative value. 2. Enter characters. 3. Update cart.	"-5", "ABC"	System should reject invalid values with validation message.	Critical
CART_QTY_006	Verify maximum quantity restriction	1. Increase quantity beyond inventory limit.	Quantity: 999	System should restrict quantity and display appropriate message.	High

4. Price Calculation Accuracy Scenarios

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_PRICE_001	Verify product price displayed correctly	1. Add product. 2. Compare product page price with cart price.	Product: Laptop	Cart price should match product listing price.	Critical
CART_PRICE_002	Verify subtotal calculation	1. Add multiple quantities. 2. Check subtotal.	Product Price: $100 Quantity: 3	Subtotal should equal $300.	Critical
CART_PRICE_003	Verify total calculation for multiple products	1. Add different products. 2. Calculate manually. 3. Compare displayed total.	Product A: $100 Product B: $200	Total should match expected calculation.	Critical
CART_PRICE_004	Verify decimal price handling	1. Add product with decimal price. 2. Check calculations.	Product Price: $19.99	System should correctly calculate decimal values.	High
CART_PRICE_005	Verify currency formatting	1. Open cart. 2. Check displayed prices.	USD / EUR / INR	Prices should display correct currency format.	Medium
CART_PRICE_006	Verify price update after product price change	1. Add product. 2. Update backend price. 3. Refresh cart.	Product price updated	Cart should follow defined pricing rules.	High

5. Tax Calculation Scenarios

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_TAX_001	Verify tax calculation for taxable products	1. Add taxable item. 2. View cart summary.	Electronics Product	Correct tax amount should be displayed.	Critical
CART_TAX_002	Verify tax calculation for non-taxable products	1. Add non-taxable item. 2. Check tax amount.	Grocery Product	Tax should not be applied if not applicable.	High
CART_TAX_003	Verify tax changes based on location	1. Change delivery location. 2. Refresh cart.	Different states/countries	Tax should update according to location rules.	High
CART_TAX_004	Verify tax calculation after quantity update	1. Add product. 2. Increase quantity. 3. Check tax.	Quantity: 1 → 5	Tax should recalculate correctly.	High

6. Coupon / Discount Code Scenarios

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_COUPON_001	Apply valid coupon code	1. Add products. 2. Enter valid coupon. 3. Click Apply.	Coupon: SAVE20	Discount should be applied successfully.	Critical
CART_COUPON_002	Apply invalid coupon code	1. Enter invalid coupon. 2. Apply coupon.	Coupon: INVALID123	Error message should be displayed.	High
CART_COUPON_003	Apply expired coupon	1. Enter expired coupon. 2. Apply.	Coupon: OLD2025	Coupon should be rejected.	High

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_COUPON_004	Verify percentage discount calculation	1. Apply percentage coupon. 2. Verify discount amount.	10% discount on $100	Discount should equal $10.	Critical
CART_COUPON_005	Verify fixed amount discount calculation	1. Apply fixed discount coupon. 2. Verify total.	$20 discount	Total should reduce by correct amount.	High
CART_COUPON_006	Verify removing applied coupon	1. Apply coupon. 2. Remove coupon.	Valid coupon	Discount should be removed and total recalculated.	Medium
CART_COUPON_007	Verify coupon eligibility rules	1. Add products below minimum value. 2. Apply coupon.	Minimum purchase $100	Coupon should not apply if conditions are not met.	High
CART_COUPON_008	Verify multiple coupon restriction	1. Apply coupon A. 2. Apply coupon B.	Two discount codes	System should allow/reject according to business rules.	Medium

7. Out-of-Stock Item Handling Scenarios

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_STOCK_001	Verify adding out-of-stock product	1. Open unavailable product. 2. Click Add to Cart.	Product Stock: 0	Add to Cart should be disabled or error message displayed.	Critical
CART_STOCK_002	Verify stock validation during checkout	1. Add product. 2. Reduce inventory externally. 3. Proceed to checkout.	Stock becomes unavailable	User should be notified before order placement.	Critical

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_STOCK_003	Verify cart message for unavailable item	1. Add product. 2. Product becomes unavailable. 3. Open cart.	Product unavailable	Cart should display stock availability warning.	High
CART_STOCK_004	Verify quantity adjustment when stock reduces	1. Add quantity 10. 2. Reduce stock to 5. 3. Refresh cart.	Available stock: 5	Quantity should adjust or prompt user.	High
CART_STOCK_005	Verify removing out-of-stock item	1. Open cart with unavailable item. 2. Remove item.	Out-of-stock product	Item should be removed successfully.	Medium

8. Performance, UI & Reliability Scenarios

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_PERF_001	Verify cart page load performance	1. Add multiple products. 2. Open cart. 3. Measure load time.	50 items	Cart should load within defined SLA.	Critical
CART_PERF_002	Verify cart performance under peak traffic	1. Simulate thousands of users. 2. Add/update carts simultaneously.	10,000 concurrent users	Cart service should remain stable.	Critical
CART_UI_001	Verify cart UI responsiveness	1. Open cart on desktop/mobile/tablet. 2. Validate layout.	Multiple devices	UI should render correctly without overlap.	Medium
CART_UI_002	Verify checkout button visibility	1. Add item. 2. Open cart.	Product in cart	Checkout button should be visible and enabled.	High

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
CART_SEC_001	Verify SQL injection prevention in cart inputs	1. Enter SQL payload in coupon/input fields. 2. Submit.	`' OR 1=1 --`	Application should sanitize input and prevent attacks.	Critical
CART_SEC_002	Verify XSS prevention in cart fields	1. Enter script payload. 2. Submit.	`<script>alert(1)</script>`	Script should not execute.	Critical

Test Coverage Summary

Category	Coverage
Item Addition	Covered
Item Removal	Covered
Quantity Management	Covered
Price Calculation	Covered
Tax Validation	Covered
Coupon & Discount Handling	Covered
Inventory / Out-of-Stock Handling	Covered
Performance Testing	Covered
Security Validation	Covered
UI & Reliability Testing	Covered

Total Test Scenarios: 50+
Test Type Coverage: Functional + Regression + Security + Performance + UI + End-to-End Validation.

You said:

Act as a Senior API Test Automation Specialist. Design API test cases for a RESTful user registration endpoint (`POST /api/v1/register`). Cover positive payload validation, negative scenarios (missing mandatory fields, invalid email format, weak passwords), duplicate registrations, and security constraints. For each test case, specify the HTTP method, headers, request body JSON, expected HTTP status code, and response body structure. Format the output cleanly using code blocks and bullet points..Provide the output in a structured Markdown table with columns: Test ID, Scenario, Test Steps, Test Data, Expected Result, and Priority."

ChatGPT said:

REST API Test Case Suite — User Registration Endpoint

Role: Senior API Test Automation Specialist
API Under Test: `POST /api/v1/register`
Purpose: Validate user registration functionality, input validation, security controls, and API reliability.

API Test Cases

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
API_REG_001	Verify successful user registration with valid payload	1. Send POST request to registration endpoint. 2. Include all mandatory fields. 3. Validate response.	**HTTP Method:** POST **Headers:** `json {  "Content-Type": "application/json",   "Accept": "application/json"  } ` **Request Body:** `json {  "firstName": "John",  "lastName": "Smith",  "email": "john.smith@test.com",  "password": "Secure@12345" } `	**Status Code:** `201 Created` Response structure: `json {  "success": true,   "message": "User registered successfully",  "userId": "12345"  } `	Critical
API_REG_002	Verify registration with minimum valid fields	1. Send request using only mandatory fields. 2. Validate response.	Body: `json { "email":"user@test.com",  "password":"Secure@12345" } `	**Status Code:** `201 Created` User should be created successfully if optional fields are not required.	High

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
API_REG_003	Verify email format validation	1. Send registration request with invalid email. 2. Validate validation response.	Body: `json { "email":"invalid-email", "password":"Secure@12345" } `	**Status Code:** `400 Bad Request` Response: `json { "error":"Invalid email format" } `	High
API_REG_004	Verify missing email field validation	1. Remove email field. 2. Send POST request.	Body: `json { "password":"Secure@12345" } `	**Status Code:** `400 Bad Request` Response should identify missing mandatory field.	Critical
API_REG_005	Verify missing password field validation	1. Remove password field. 2. Send request.	Body: `json { "email":"test@test.com" } `	**Status Code:** `400 Bad Request` Password required validation message should be returned.	Critical
API_REG_006	Verify empty mandatory fields validation	1. Send request with empty values. 2. Validate response.	Body: `json { "email":"", "password":"" } `	**Status Code:** `400 Bad Request` Empty fields should not be accepted.	High
API_REG_007	Verify password minimum length validation	1. Send password shorter than allowed limit. 2. Validate response.	Password: `Abc@1`	**Status Code:** `400 Bad Request` Password length validation message should appear.	High

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
API_REG_008	Verify weak password rejection	1. Submit weak password. 2. Validate API response.	Password: `password123`	**Status Code:** `400 Bad Request` Password complexity rules should be enforced.	High
API_REG_009	Verify password without special character	1. Send password without special characters. 2. Validate response.	Password: `Password123`	**Status Code:** `400 Bad Request` Special character requirement validation should trigger.	Medium
API_REG_010	Verify password without uppercase character	1. Submit lowercase-only password. 2. Validate response.	Password: `password@123`	**Status Code:** `400 Bad Request` Uppercase character requirement should be enforced.	Medium
API_REG_011	Verify duplicate email registration	1. Register user successfully. 2. Register same email again. 3. Validate response.	Existing Email: `john@test.com`	**Status Code:** `409 Conflict` Response: `json { "error":"User already exists" } `	Critical

API_REG_012	Verify duplicate registration with different case email	1. Register using lowercase email.
		2. Register using uppercase email.
			Emails:
			`test@test.com`
			`TEST@TEST.COM`
				Status Code: `409 Conflict`
				Email uniqueness should be case insensitive.
					High

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
API_REG_013	Verify registration with additional unknown fields	1. Add unsupported fields. 2. Submit request.	Body: `json { "email":"test@test.com", "password":"Secure@12345", "role":"admin" } `	**Status Code:** `400 Bad Request` or ignored based on API contract. No unauthorized fields should be processed.	Medium
API_REG_014	Verify invalid JSON payload handling	1. Send malformed JSON. 2. Validate response.	Body: `json { "email":"test@test.com" } `	**Status Code:** `400 Bad Request` Invalid JSON error should be returned.	High
API_REG_015	Verify missing Content-Type header	1. Remove Content-Type header. 2. Send request.	Headers: `json { "Accept":"application/json" } `	**Status Code:** `415 Unsupported Media Type`	Medium
API_REG_016	Verify unsupported HTTP method	1. Send GET request instead of POST. 2. Validate response.	Method: `GET /api/v1/register`	**Status Code:** `405 Method Not Allowed`	Medium
API_REG_017	Verify SQL injection protection	1. Send SQL payload in input fields. 2. Monitor response.	Email: `text ' OR 1=1 -- `	**Status Code:** `400 Bad Request` Input should be sanitized. No database information exposed.	Critical
API_REG_018	Verify XSS attack prevention	1. Send script payload. 2. Validate response.	First Name: `html <script>alert('XSS')</script> `	**Status Code:** `400 Bad Request` or sanitized response. Script must not execute.	Critical

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
API_REG_019	Verify excessively long input handling	1. Send extremely large values. 2. Validate response.	Email length: 5000 characters	**Status Code:** `400 Bad Request` API should handle gracefully without crashing.	High
API_REG_020	Verify special character handling in name fields	1. Submit invalid characters. 2. Validate response.	Name: `@@@@123`	**Status Code:** `400 Bad Request` Validation message should be returned.	Medium
API_REG_021	Verify authentication is not required for registration	1. Call API without Authorization header. 2. Validate response.	Headers: `json { "Content-Type":"application/json" } `	**Status Code:** `201 Created` Registration should work without authentication.	High
API_REG_022	Verify response schema validation	1. Register valid user. 2. Validate response fields.	Valid registration payload	Response should contain expected fields: `json { "success":true, "message":"string", "userId":"string" } `	High
API_REG_023	Verify API response time	1. Send valid registration request. 2. Measure response time.	Valid user payload	API should respond within agreed SLA (example: <2 seconds).	High
API_REG_024	Verify concurrent registration handling	1. Send multiple registration requests simultaneously. 2. Monitor results.	1000 concurrent requests	API should handle load without duplicate user creation or failures.	Critical

Test ID	Scenario	Test Steps	Test Data	Expected Result	Priority
API_REG_025	Verify database transaction rollback on failure	1. Trigger registration failure after validation. 2. Check database state.	Invalid registration payload	No partial user record should be created.	High

---

Sample API Request Format

HTTP Request

```
POST /api/v1/register HTTP/1.1
Host: example.com
Content-Type: application/json
Accept: application/json
```

Request Body

```
{
  "firstName": "John",
  "lastName": "Smith",
  "email": "john.smith@test.com",
  "password": "Secure@12345"
}
```

Expected Success Response

```
{
  "success": true,
  "message": "User registered successfully",
  "userId": "12345",
  "timestamp": "2026-10-17T10:00:00Z"
}
```

HTTP Status: `201 Created`

Expected Validation Error Response

```
{
  "success": false,
  "errorCode": "VALIDATION_ERROR",
  "message": "Invalid request data",
  "errors": [
    {
      "field": "email",
      "message": "Invalid email format"
    }
  ]
}
```

HTTP Status: `400 Bad Request`

Coverage Summary

Testing Area	Coverage
Positive Registration Flow	✅ Covered
Mandatory Field Validation	✅ Covered
Email Validation	✅ Covered
Password Policy Validation	✅ Covered
Duplicate User Prevention	✅ Covered
API Contract Validation	✅ Covered
SQL Injection Protection	✅ Covered
XSS Protection	✅ Covered
Performance Testing	✅ Covered
Concurrent User Registration	✅ Covered

Total API Test Scenarios: 25

Test Coverage Type: Functional + Negative + Security + Performance + API Contract Validation.